
FINITE POSSIBILITY MECHANICS

MODULAR PAPER 02

Executable FPM

Reference Runtime, Numerical Contract, and Reproducible Audit

COMPUTATIONAL PAPER

Alx Spiker

July 2026

Edmonton, Alberta, Canada

Part of the FPM modular research series

Contents

Modular Paper 02 in the Finite Possibility Mechanics research series

Abstract	3
1. Executable specification	3
1.1 Scope of the reference runtime	3
1.2 Source boundary	3
2. Geometry and state	4
2.1 Periodic cubic lattice	4
2.2 Per-node state	4
2.3 Standard initial condition	4
3. Ordered update pipeline	4
3.1 Tick sequence	5
3.2 Interior replenishment	5
3.3 Capacity resolver	5
4. Numerical realization	6
4.1 Why the reference uses floating point	6
4.2 Residual interpretation	6
4.3 Reproducibility controls	6
5. Conservation audit	6
5.1 Focused local bridge	6
5.2 Master-chain ledger	7
6. Validation suite	7
7. Reproduction protocol	8
7.1 Standard commands	8
7.2 Expected artifacts	8
7.3 Reproduction criterion	8
8. Regression classification	9
8.1 Invariants	9

8.2 Geometry-dependent observables	9
8.3 Bridge-dependent observables	9
9. Conclusion	9
References	10

Abstract

This paper specifies the public Python reference implementation of Finite Possibility Mechanics. The runtime executes the FPM master chain on a periodic $5 \times 5 \times 5$ cubic lattice: 125 nodes, six nearest neighbours per node, and 400 ordered ticks in the standard production audit. Each node carries stored energy, a directed 3×3 route ledger, a nine-channel complex carrier, viscosity, prior, cache bias, and derived observables. Ordinary action is replenished through the local reversible kernel proved in Paper 1; capacity overflow, starvation, Landauer debit, and explicit torsion-link operations are recorded in separate signed accounts.

The implementation reproduces the exact algebraic identities to floating-point precision. The local bridge audit reports row-stochastic error 1.11×10^{-16} , detailed-balance error 2.78×10^{-17} , nodewise continuity residual 6.94×10^{-17} , exact local energy edge-flux antisymmetry, and equilibrium L1 error 7.28×10^{-14} . The 400-tick master run closes the expanded ledger to 6.40×10^{-14} while exercising capacity routing, starvation, information erasure, finite microcell quantization, and linked torsion quantization. These results establish a reproducible executable specification whose numerical residuals are many orders below the modeled action scales.

Computational result

The public runtime implements the finite periodic cubic geometry and local replenishment theorem directly. Floating-point residuals quantify numerical representation; they are not unrecorded energy channels.

1. Executable specification

A mathematical model becomes operational only when every state variable, update order, boundary branch, and observable has an executable meaning. The reference simulator serves that role for FPM. It is deliberately transparent: all constants are constructed in one derivation object, all experiments return machine-readable records, every chart is generated from those records, and output paths are explicit.

1.1 Scope of the reference runtime

- Instantiate the finite graph, directed ledger, carrier, viscosity, action, and signed energy accounts.
- Evaluate the master chain tick by tick with a fixed seed.
- Run theorem checks and physical correspondence diagnostics as separate named experiments.
- Write JSON results to `outputs/` and charts to `simulator_charts/` by default.
- Provide a readable real-valued reference against which an exact integer engine can be compared.

1.2 Source boundary

The executable specification is contained in `scripts/fpm_simulator.py`, with independent algebraic checks in `scripts/verify_derivations.py` and the focused local-transport harness `scripts/fpm_local_energy_bridge.py`. The generator and paper consume their outputs; they do not silently recompute alternative results.

This executable scope includes Paper 1's stochastic kernel, detailed balance, local and regional energy continuity, finite propagation cone, activity-weighted equilibrium, and expanded signed ledger. It does not yet construct an edge 1-cochain φ , compute a plaquette coboundary $A=d\varphi$, or audit periodic cohomology sectors. Consequently, antisymmetric internal route matrices and explicit torsion links are bridge ansatzes here, not an executable proof of Paper 1's conditional exact-cochain theorem. Paper 5 must supply and test that map.

2. Geometry and state

2.1 Periodic cubic lattice

$$\Lambda_5 = (Z/5Z)^3, \quad |\Lambda_5|=125, \quad \deg(i)=6$$

Coordinates are enumerated modulo five. The neighbour map adds and subtracts one unit on each axis with periodic wraparound. Side length five is large enough that the positive and negative neighbour on every axis is distinct. The same six-neighbour map is supplied to ordinary replenishment and local overflow routing, so the geometry does not change between the interior and capacity branches.

2.2 Per-node state

$$X_{i,t} = (E, R, \psi, b, \tau, \pi, \Omega_{\text{prev},i,t})$$

E is stored energy; R is the 3×3 directed route ledger; ψ in C^9 is the normalized carrier; b is cache bias; τ is the local truth target; π is the fallback prior; and Ω_{prev} stores the preceding viscosity for the smoothness cost. The symbols τ and π are lowercase and match Paper 1 and the runtime fields τ and π ; π is not the equilibrium vector p^{eq} . The current viscosity Ω , the binary projections p_L and p_R , and coherence c are derived from this stored state rather than evolved as independent variables.

Paper 2 names these carrier fields because the unified runtime must allocate and update them in one transaction. Their normalization constraints, phase dynamics, low-energy consolidation, finite microcell rule, and Landauer accounting are derived in Paper 3. Until that dependency is established, the present paper treats those routines as declared downstream modules and audits only their executable interfaces and ledger effects.

2.3 Standard initial condition

The production run uses pseudorandom seed 17 and initializes 125 nodes near the balanced carrier state. Each node starts with 0.5 normalized energy units. Because the derived ceiling is $E_{\text{max}}=2/3$, this is 75% of capacity and gives total initial stored energy 62.5. The run begins near the normal operating viscosity, and all perturbations are small and reproducible. Torsion links are created as explicit node pairs rather than inferred from distance.

3. Ordered update pipeline

One tick is an ordered transaction. Later operations consume the state written by earlier operations, implementing Axiom A4 directly.

3.1 Tick sequence

- Compute route-ledger invariants, normalized entropy and balance, spectral weights, capacity, and viscosity.
- Evaluate the declared semantic and geometric action terms plus the viscosity-smoothness term.
- If action is unaffordable, pay what is available and record the remainder as starvation deficit.
- Construct activity weights and the six-neighbour local kernel; apply $r=P^T L$.
- Route positive capacity overflow to adjacent available capacity; record unabsorbed remainder as exhaust.
- Rotate each carrier channel by its route-cost phase and refresh scalar projections.
- Apply low-energy consolidation, finite microcell selection, and Landauer debit when the gate is active.
- Apply explicitly linked joint-torsion operations through their separate branch.
- Commit state and append stored-energy, expanded-ledger, action, viscosity, dispersion, and event diagnostics.

3.2 Interior replenishment

$$r_t = P_t^T L_t, \quad E_{\text{raw},t+1} = E_t - L_t + r_t$$

The simulator constructs P from the exact formula in Paper 1 and validates the matrix before use. The transpose is intentional: rows describe where each source sends its paid action, while $P^T L$ collects what each destination receives.

3.3 Capacity resolver

For $E_{\text{raw}} > E_{\text{max}}$, the excess is offered only to the same six neighbouring sites, weighted by their available capacity. Concurrent proposals are accumulated and limited by receiving capacity. Any unaccepted remainder enters thermal exhaust. For $E_{\text{raw}} < 0$, the state is clipped to zero and the unpaid amount enters starvation deficit. These events alter stored energy but close the expanded ledger exactly in real arithmetic.

In the notation of Paper 1, exhaust and paid erasure increment the local positive event ledger X_i , while an unpaid zero-boundary amount increments the local deficit ledger D_i . The runtime retains the site-resolved increments for each tick and then forms the observer totals $X(t) = \sum_i X_i(t)$ and $D(t) = \sum_i D_i(t)$ used by Theorem 8. Summation occurs after the local event; it is an audit reduction, not a propagating signal.

4. Numerical realization

4.1 Why the reference uses floating point

The Python implementation uses IEEE-754 double precision [3] and NumPy complex arrays because it is the clearest representation of the equations. It preserves direct correspondence between source expressions and the manuscript, supports singular-value decomposition and phase rotation naturally, and makes sensitivity analysis inexpensive. This is a reference semantics, not a claim that physical state is binary floating-point data.

Tensorless is not assumed to be this runtime with floating point replaced by integers. Its synchronous sandbox receives caller-translated action and payload values, resolves contention with exact integer policies, and currently replenishes through a global proportional allocator. That allocator realizes an equilibrium-style activity weighting, not the ordinary one-tick local kernel $r=P^T L$ used here. Tensorless also reports aggregate exhaust and starvation counters rather than this runtime's site-resolved event history. Paper 4 specifies those differences and limits cross-runtime comparisons to explicitly shared contracts.

4.2 Residual interpretation

An algebraic identity that is exact over real numbers may leave a residual near 10^{-16} after a sequence of floating-point additions and multiplications. The correct test is scale-aware: compare the residual with machine precision and the magnitude of the summed quantities, then verify convergence or exact agreement in an integer realization. The reference audit records residuals instead of rounding them to zero in the output.

Numerical contract

Every reported residual is an observable of the implementation. PASS means the stated criterion was met; it does not rewrite the measured residual as mathematical zero.

4.3 Reproducibility controls

- Fixed random seeds for stochastic initialization and theorem trials.
- Deterministic lattice enumeration and neighbour ordering.
- Named output directories, overridable through environment variables.
- JSON serialization of axioms, derived values, theorem records, bridge records, experiments, and the master trajectory.
- Charts regenerated from the same in-memory result object written to JSON.

5. Conservation audit

5.1 Focused local bridge

The independent local-energy harness checks the kernel on non-uniform positive weights and viscosities. Its current results are:

- row-stochastic residual: 1.11×10^{-16}
- stationary-weight residual: 6.94×10^{-18}
- detailed-balance residual: 2.78×10^{-17}
- global replenishment residual: 3.55×10^{-15}
- nodewise continuity residual: 6.94×10^{-17}
- local energy edge-flux antisymmetry residual: exactly 0 in the evaluated representation
- support radii after ticks 0 through 8: 0,1,2,3,4,5,6,7,8
- equilibrium L1 residual: 7.28×10^{-14}

5.2 Master-chain ledger

The 400-tick production trajectory begins with total stored energy 62.5. Once boundary and special branches activate, stored energy changes; this is expected. The expanded ledger remains 62.5 to a maximum residual of 6.40×10^{-14} . The run records 67.6694 units of locally routed spillover, 16.1690 units of external thermal exhaust, 23.2968 units of starvation deficit, and 1.7619×10^{-7} units of Landauer debit. It also executes 3,842 finite microcell quantizations and 1,921 linked torsion quantizations.

Ledger reading

The simulator does not conserve stored daemon energy at every tick. It conserves ordinary interior replenishment locally and the expanded signed ledger across all declared branches.

6. Validation suite

The suite contains sixteen primary experiments plus one starvation subtest. Their roles differ: some test mathematical invariants, some test numerical implementation, and some evaluate physical bridge formulae. The domain-specific names below are regression labels for downstream correspondences developed in Papers 5 and 6; understanding how a route ledger is mapped to dephasing, galaxy data, or a Bell test is not a prerequisite for auditing the runtime contract in this paper. Paper 6 supplies the full evidence hierarchy. The current runtime records the following headline results.

- Dispersion contraction: 0 violations in 6,000 evaluated updates.
- Lindblad correspondence: RMSE 1.79×10^{-102} for the selected dephasing map.
- Closed-universe conservation: final floating-point drift 7.96×10^{-14} percent.
- Spectral-gap weights: isotropic leading weight 1/3.
- Mean-field truth closure: final mismatch 0.02637 in the declared convergence experiment.
- Point-Pair iteration: the fixed-point equation closes with zero residual in the evaluated binary64 arithmetic.
- Bounded depletion: effective floor 0.0314 at load 10^6 .
- Semantic-entropy ledger: saturated closure in the specified erasure transaction.
- Wrong-lock starvation subtest: starvation begins at tick 1.

- Finite lag construction: ratio 31.8738629472.
- SPARC gas-boundary bridge: archived benchmark metadata reports median RMSE 11.61 km/s; the live rerun is marked unavailable when the external tables are absent.
- Exact carrier capacity: 1,452,997,909 bit-equivalent slots.
- Born finite allocation: maximum total-variation error 1.20×10^{-9} .
- Joint torsion CHSH: $S=2.8284271261$ in the implemented joint rule.
- Runtime linked quantization: the linked pair enters one joint ledger operation.
- Bare coupling construction: inverse coupling 136.7946397586.
- Local replenishment bridge: maximum continuity residual 6.94×10^{-17} .

7. Reproduction protocol

7.1 Standard commands

```
python3 scripts/verify_derivations.py
```

```
python3 scripts/fpm_local_energy_bridge.py
```

```
python3 scripts/fpm_simulator.py
```

The simulator creates the output directory if required. `FPM_OUTPUT_DIR` redirects JSON files and `FPM_SIMULATOR_CHARTS_DIR` redirects charts without changing the equations. A valid reproduction retains the emitted JSON, console summary, environment, Python and dependency versions, and source commit identifier.

7.2 Expected artifacts

- `outputs/verification_results.json`: focused derivation and local-bridge checks.
- `outputs/fpm_results.json`: complete axioms, constants, theorem records, bridges, experiments, and trajectory.
- `simulator_charts/*.png`: plots generated from the same run.

7.3 Reproduction criterion

Exact decimal equality is required for integer and rational constructions. Floating-point arrays are compared using declared absolute or relative criteria appropriate to their scale. Geometry-dependent stochastic trajectories must reproduce under the same seed and software stack; cross-platform runs should reproduce all classifications and agree within the stated numerical tolerance.

8. Regression classification

8.1 Invariants

Row sums, local energy-flux antisymmetry, signed-ledger identity, neighbour count, carrier normalization, exact integer capacity, and support-cone growth are structural invariants. A geometry change does not license these to drift. This energy-flux test is distinct from the conditional exact-cochain construction for directed asymmetry in Paper 1.

8.2 Geometry-dependent observables

Mixing time, transient stored-energy curves, event counts, mean viscosity, mean action, and spatial correlation patterns depend on lattice size and adjacency. The upgrade from a one-dimensional ring to the 125-node cubic lattice legitimately changed these outputs. Regression tests therefore compare them with geometry-matched baselines rather than forcing historical values to persist.

8.3 Bridge-dependent observables

Galaxy, CMB, Bell, and coupling results depend on bridge definitions in addition to the runtime core. They are reported in the same executable package for traceability, but a change in one bridge must not be interpreted as a failure of local conservation unless the relevant foundational invariant also changes.

9. Conclusion

Executable FPM is a transparent real-valued implementation of the finite local mechanics defined in Paper 1. It runs the intended six-neighbour cubic geometry, uses the same adjacency for ordinary replenishment and local overflow, records every non-interior energy branch, and exposes both stored energy and the expanded closed ledger. The numerical results confirm the exact algebra at machine precision while preserving the measured residuals.

This reference now provides the common executable target for two complementary developments: Paper 3's finite carrier and information dynamics, and Paper 4's deterministic exact-integer realization. The Python runtime prioritizes mathematical visibility; the exact engine prioritizes arithmetic closure and operational determinism. Agreement between them is the strongest implementation audit available to the framework.

Executable summary

The public runtime is not a diagram of the theory. It is the theory's finite local update chain, executed on 125 six-neighbour nodes and audited transaction by transaction.

References

- [1] A. Spiker, Finite-Possibility-Mechanics-Complete, source repository and reproducible runtime.
<https://github.com/alxspiker/Finite-Possibility-Mechanics-Complete>
- [2] A. Spiker, FPM Foundations: Finite Local Substrates, Directed Route Ledgers, and Exact Closure, Modular Paper 01.
- [3] IEEE Computer Society, IEEE Standard for Floating-Point Arithmetic, IEEE 754-2019.
<https://doi.org/10.1109/IEEESTD.2019.8766229>
- [4] N. Metropolis et al., "Equation of State Calculations by Fast Computing Machines," Journal of Chemical Physics 21, 1087–1092 (1953). <https://doi.org/10.1063/1.1699114>